

SHREK-HRM: Self-Correcting Hierarchical Reasoning Models

Thomas Heim
OsloMet
thhei7342@oslomet.no

Max Dyrø Finanger
OsloMet
madyr3382@oslomet.no

Sebastian Skrøvseth Haugen
OsloMet
sehau5609@oslomet.no

Abstract—This paper explores research into Green AI, Hierarchical Reasoning Models (HRM), and Tiny Recursive Models (TRM). We have introduced the Stagnation-aware Halting and Reasoning Error Knowledge Hierarchical Reasoning Model (SHREK-HRM). Based upon the original HRM, with additional flip-rate detector, learned error estimator and stagnation control. The results show that the SHREK-HRM can outperform the original HRM on several benchmarks while improving the number of gigaflops (GFLOPS) needed for operations.

Index Terms—Hierarchical Reasoning Models (HRM), Tiny Recursive Models, Green AI, Stagnation-aware Halting and Reasoning Error Knowledge Hierarchical Reasoning Model (SHREK-HRM)

I. INTRODUCTION

Recent advances in artificial intelligence have been driven by the increase in the scale of large language models (LLMs), increasing data size, parameters, and training set. Despite these successes, it remains unclear whether increasing model scale alone is sufficient to produce robust reasoning and reliable generalization. Recent research suggests that improvements in benchmark performance do not necessarily correspond to deeper understanding or structured reasoning capabilities [1], [2].

From a cognitive science perspective, Cuskley, Woods, and Flaherty [1] argue that LLMs exhibit linguistic fluency without genuine semantic understanding, as their behavior is primarily driven by statistical pattern learning rather than grounded conceptual representations. Similarly, Lake and Baroni [3] demonstrate that neural models struggle with systematic compositional generalization, performing well on familiar inputs while failing to recombine known components in novel contexts.

Additional empirical studies highlight limitations in multi-step reasoning and planning tasks. Valmeekam et al. [2] show that LLM performance degrades significantly as reasoning depth increases, suggesting that scaling model parameters does not necessarily resolve fundamental reasoning challenges. Together, these findings suggest that the limitations of large models may be architectural rather than purely quantitative.

At the same time, concerns have emerged about computational cost and sustainability. The concept of "Green AI" [4] emphasizes the need to consider efficiency and resource consumption alongside performance. This perspective motivates us to explore alternative architectures that can achieve strong reasoning capabilities without relying on extreme model scale.

Recent research has therefore started to investigate smaller, structured reasoning models that rely on iterative computation rather than parameter growth. Hierarchical Reasoning Models (HRM) [5] and Tiny Recursive Models (TRM) [6] demonstrate that compact architectures can achieve competitive results on symbolic reasoning benchmarks. However, subsequent analysis [7] reveals that these models can be unstable during inference, including convergence to incorrect fixed points and behavior that resembles guessing rather than structured reasoning.

A limitation of the existing hierarchical reasoning models is the lack of internal self-correcting mechanisms. This means that when a model converges to a wrong state they cannot recover or detect the errors which leads to unreliable reasoning. It is not just about better architecture, but a more dynamic model that can change and adjust its reasoning.

Looking at these limitations, this work investigates whether hierarchical reasoning architectures can be improved to produce more stable and reliable reasoning behavior while maintaining computational efficiency. We introduce a modified hierarchical reasoning framework, the **Stagnation-aware Halting and Reasoning Error Knowledge Hierarchical Reasoning Model (SHREK-HRM)**, that incorporates mechanisms for early logical verification and the detection of reasoning cycles during inference.

The main contributions of this work are summarized as follows:

- We analyze the limitations of existing hierarchical reasoning models, particularly the "guessing" phenomenon and the easy-task paradox.
- We propose a self-correcting hierarchical reasoning architecture that introduces flip-rate monitoring, error injection, and stagnation-aware halting during inference.
- We evaluate the proposed approach against existing HRM and TRM models on symbolic reasoning benchmarks.
- We analyze computational efficiency and reasoning stability to assess whether compact reasoning models can provide a viable alternative to large-scale language models.

A. Research Question:

Can hierarchical reasoning architectures be further enhanced with self correction mechanisms, improving their reasoning stability, and efficiency without increasing the model size?

B. Paper Organization:

The paper is organized as follows. Section II reviews the relevant literature on hierarchical reasoning models and efficiency-driven AI approaches and presents the identified research gaps. Section III presents the proposed SHREK-HRM architecture. Section IV describes the experimental methodology. The results follow in Sections V and the discussion in VI.

II. BACKGROUND

A. Green AI

Schwartz et al. [4] highlight the growing imbalance between performance improvements and computational cost in AI research. The authors distinguish between two paradigms: "Red AI," which prioritizes maximizing benchmark performance regardless of resource consumption, and "Green AI," which emphasizes efficiency, accessibility, and reproducibility as core evaluation criteria.

Rather than proposing a new model, the article introduces a framework for reporting training cost, energy consumption, and efficiency metrics along with precision. This frames progress in machine learning by encouraging researchers to consider computational resources as part of model evaluation.

The main contribution of the work is the efficiency-aware evaluation in AI research. However, the study does not propose concrete architectural alternatives for achieving efficient reasoning, leaving open the question of how to design models that maintain performance under strict computational constraints.

B. Hierarchical Reasoning Model

The first paper that introduced the hierarchical reasoning model architecture addresses the research problem of "paradoxical shallowness" in large language models [5]. This means that models can often be limited by rigid computational depth that prevent models from solving problems that require polynomial time [5]. The authors argue that modern solutions like Chain-of-Thought (CoT) are too slow and computationally heavy as it generates large number of tokens for complex tasks. The objective is to make the Hierarchical Reasoning Model (HRM) which performs deep reasoning within an internal hidden, called latent reasoning [5].

The hierarchical reasoning model uses a recurrent architecture consisting of two modules: a high-level recurrent module for abstract planning and a low-level recurrent module for detailed computations. The hierarchical reasoning model uses what the authors call hierarchical convergence, where the low-level recurrent module resets its state each time the high-level module updates, making sure that it continues to have high computational activity over many steps [5]. The authors did the experiments using a 27-million parameter model on complex benchmarks such as ARC-AGI, Sudoku-Extreme, and 30x30 Maze-Hard navigation. They used small-sample learning using 1,000 training examples and no pre-training or external CoT data. The hierarchical reasoning model achieved 40.3% accuracy on ARC-AGI-1 and 5% on ARC-AGI-2 and

near-perfect results for Sudoku-Extreme (9x9) and Maze-Hard (30x30), outperforming all the other models [5].

HRM's novelty lies in its capability to achieve inference-time-scaling which is a process where the model's reasoning accuracy improves by increasing the number of internal recurrent "thinking" steps during testing [5]. A key contribution is the Adaptive Computational Time (ACT) mechanism, which uses a Q-learning policy to decide at each step whether to continue reasoning or stop and output the result. This makes the model more efficient so that it can use longer time on complex tasks and a shorter time on easier task [5]. Additionally, the researchers introduce a one-step gradient approximation for training, where this method optimizes the model only in its equilibrium state. This is the point where the internal thoughts settle on a stable final answer, replacing memory-heavy Backpropagation Through Time [5].

One limitation of this study is that the comparison of the models might not be fair. The HRM model is a highly specified model trained for this specific task, compared against transformers that are more generalized.

C. Less is More: Recursive Reasoning with Tiny Networks

The core research problem addressed in [6] is the struggle of LLMs with hard tasks that require deep reasoning. LLMs rely heavily on Chains of Thought (CoT), but this is often computationally expensive and data-hungry [6]. The authors [6] focus on improving the Hierarchical Reasoning Model (HRM), which has shown promising results but relies on complex biological arguments and mathematical theories that might not always be applicable. The objective is to propose the Tiny Recursive Model (TRM), which is a much simpler and more parameter-efficient architecture that achieves better generalization only using a tiny network [6].

The tiny recursive model architecture is simpler than the Hierarchical Reasoning Model architecture by only using a single network with only two layers that recursively improve the predicted answers through multiple supervision steps [6]. The methodology involves a latent recursion process where a latent feature is updated given the question and current answer, followed by refining the answer itself. The 7M parameter model performed the best on benchmarks like Sudoku-Extreme, Maze-Hard, and ARC-AGI, achieving 45% on ARC-AGI-1 and 8%, outperforming the original HRM-model [6].

The novelty of the tiny recursive network lies in the approach of trying to improve HRM by removing the fixed-point theorem requirement and instead using 1-step gradient approximations, it back propagates through the full recursion process, which boosts generalization [6]. In addition to this the paper gives a clearer view of how the model reasons by using two internal features. The first feature is the current solution (y), which is the best guess at the answer so far and the reasoning feature (z), which we can think out as a "mental scratchpad" or working memory [6].

The main strength for the tiny recursive model is that it is very efficient on symbolic reasoning tasks and performed better than HRM. On the other hand one can argue that the

claim that less is more here can be attributed to the fact that a smaller model has fewer chances for overfitting compared to larger models.

D. Are Your Reasoning Models Reasoning or Guessing?

Ren and Liu [7] provide a mechanistic analysis of HRM. Through Principal Component Analysis of the model’s internal trajectory on Sudoku-Extreme, they show that HRM frequently converges to incorrect fixed points and exhibits oscillating reasoning rather than stable convergence to the correct answer. They also identify the “easy-task paradox”, where the model solves harder puzzles but fails on simpler ones because it gets trapped in early fixed points. The authors mitigate these issues with data augmentation, input perturbation, and model bootstrapping, raising Sudoku-Extreme accuracy from 54.5% to 96.9%. A limitation is that their evaluation is restricted to Sudoku-Extreme, and that their mitigation’s rely on training time improvements rather than structural changes to the model.

E. Training Compute-Optimal Large Language Models

Hoffmann et al. [8] investigate how computational resources should be optimally allocated when training large language models. Traditional scaling practices often increase the number of parameters without proportionally increasing training data, leading to inefficient use of compute.

The authors conduct empirical scaling experiments where the total training compute, measured in floating-point operations (GFLOPS), is kept constant while the model size and the dataset size are varied. Their results show that many existing LLMs are under trained relative to their size and that performance depends on the balance between parameters and training tokens.

The study introduces compute-optimal scaling laws and demonstrates that smaller models trained on more data can outperform larger models trained sub-optimally. A limitation is that the evaluation focuses on next-token prediction tasks rather than complex reasoning benchmarks.

F. Research Gaps:

The gaps identified in the research above are the lack of self-correction, and the missing real-world cost of models.

- Gap 1: Lack of Self-Correction in Hierarchical Reasoning Models: Mechanistic analysis has shown that HRM models can get stuck on wrong answers or oscillate between incorrect solutions during inference. Current architectures have no mechanism to detect internal errors or adjust their reasoning based on self-feedback.
- Gap 2: Missing Real World Cost Comparisons: No prior work has compared HRM and TRM architectures on identical hardware in terms of inference GFLOPS, average reasoning steps, and accuracy.

III. RESEARCH OBJECTIVE, AND PROBLEM STATEMENT

A. Research Objective:

To investigate possible improvements in HRM algorithms, we want to avoid high computational cost and add self-

correction to avoid issues common to HRM models. Additionally, we test a new SHREK-HRM algorithm that introduces mechanisms for escaping loops with self-correction and adding more layers to the hierarchy. This experiment aims to compare our new algorithm with previous HRM and TRM models on even benchmarks and to compare their results and approximate computational and environmental costs. Following the principles of green AI and benchmarking best practices.

B. Problem Statement:

Recent advances in artificial intelligence have largely relied on scaling model parameters and training data, resulting in increasingly computationally intensive systems. While hierarchical reasoning models such as HRM and compact recursive architectures such as TRM demonstrate that smaller models can achieve competitive performance on symbolic reasoning tasks, recent analyzes reveal that these systems may not perform genuine structured reasoning. Instead, they often converge to incorrect fixed points, exhibit unstable inference dynamics, or rely on behavior reminiscent of probabilistic guessing.

IV. METHODOLOGY

A. Proposed Solution

SHREK-HRM extends the original Hierarchical Reasoning Model (HRM) with two lightweight self-correction mechanisms while preserving its high-level / low-level recurrent structure ($\mathbf{z}_H, \mathbf{z}_L$). All changes were implemented by forking the official HRM codebase [5].

1) Error injection. A scalar error signal is computed after each reasoning step and added back into the high-level state $\mathbf{z}_H$, nudging the model away from unstable trajectories. The signal combines two complementary sources:

- *Flip-rate:* the per-sample fraction of output tokens that changed from the previous reasoning step:

$$\text{flip_rate}_b = \frac{1}{L} \sum_{i=1}^L \mathbb{I}(\hat{y}_{b,i}^{(t)} \neq \hat{y}_{b,i}^{(t-1)}),$$

where $\hat{y}_b^{(t)}$ is the argmax prediction for sample b at step t . A high flip-rate indicates oscillation; a near-zero flip-rate indicates convergence. The signal is task-agnostic and requires no learning.

- *Learned error estimator* — a linear layer (`hidden_size` $\rightarrow$ 1) that reads the mean-pooled content tokens of $\mathbf{z}_H$ and outputs a sigmoid score in $[0, 1]$, trained with an auxiliary MSE loss against the per-sample language-model loss (min-max normalized within each batch). This component learns to recognize “confidently wrong” states that flip-rate alone cannot detect.

The combined error a 50/50 blend of the two sources is encoded to a hidden vector and injected:

$$\mathbf{z}_H \leftarrow \mathbf{z}_H + \alpha \cdot \text{error_encoder}(\text{error}),$$

where α is a warmed-up coefficient ($0 \rightarrow 0.01$).

2) Stagnation-aware Q-head. The Q-head input is augmented with a stagnation delta measuring how much $\mathbf{z}_H$ changed during the current reasoning step, allowing it to distinguish stable convergence from being stuck on a wrong answer. Concretely, the Q-head reads $[\mathbf{z}_H[:, 0], \|\Delta\mathbf{z}_H\|/\|\mathbf{z}_H\|]$, where the relative norm makes the signal scale-invariant.

The total training objective combines the LM loss, the two Q-head losses, and the auxiliary estimator loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{lm}} + 0.5(\mathcal{L}_{\text{q-halt}} + \mathcal{L}_{\text{q-continue}}) + 0.1 \cdot \mathcal{L}_{\text{aux}}.$$

We implement two variants that share the same codebase and differ only in depth, summarized in Table I.

TABLE I
SHREK-HRM MODEL VARIANTS. BOTH SHARE THE SAME ARCHITECTURE AND CODEBASE, DIFFERING ONLY IN DEPTH.

Model	Params	Layers (H+L)
SHREK-Large	~27M	4+4
SHREK-Small	~14M	2+2

B. Experiments

Datasets. We evaluate on two symbolic reasoning benchmarks: Sudoku-Extreme-1k-aug-1000 and Maze-30×30-hard-1k.

Training setup. All four models (Original HRM, TRM Attention, SHREK-Large, SHREK-Small) were trained on a single NVIDIA GH200 GPU on the Simula Research Laboratory cluster, using identical hyperparameters: learning rate 1×10^{-4} , global batch size 768 for Sudoku-Extreme and 128 for Maze-Hard, 40,000 epochs (Sudoku-Extreme) or 20,000 epochs (Maze-Hard), and evaluation every 1,000 steps. SHREK and TRM Attention use EMA weight averaging ($\mu = 0.999$); Original HRM is trained without EMA, following the original recipe. The software stack consists of PyTorch with FlashAttention and einops, the full dependencies are listed in `requirements.txt`. Training was monitored via Weights & Biases.

Evaluation. We report single-checkpoint test accuracy (all.exact_accuracy from the best checkpoint of each run). Average ACT reasoning steps are read from the Q-head halt logits. Inference GFLOPS are measured with PyTorch’s FlopCounterMode over 1,000 test puzzles, using the final checkpoint of each training run.

V. RESULTS

A. Sudoku-Extreme

Table II reports single-checkpoint accuracy on Sudoku-Extreme.

SHREK-Large (~65%) outperforms Original HRM (53%) by 12 percentage points on the dataset. SHREK-Small (~63%), with only 14M parameters, surpasses the 27M-parameter Original HRM. TRM Attention (~70%) achieves the highest accuracy on Sudoku. All the paper baselines were reproduced within stated variance, confirming a valid comparison.

TABLE II
SINGLE-CHECKPOINT TEST ACCURACY ON SUDOKU-EXTREME.

Model	Params	Accuracy
TRM Attention	~7M	~70%
SHREK-Large(Ours)	~27M	~65%
SHREK-Small(Ours)	~14M	~63%
Original HRM	~27M	53%

B. Maze-Hard

Table III reports results on Maze-Hard.

TABLE III
SINGLE-CHECKPOINT TEST ACCURACY ON MAZE-HARD.

Model	Params	Accuracy
TRM Attention	~7M	~87%
SHREK-Large (Ours)	~27M	~83%
Original HRM	~27M	~75%
SHREK-Small(Ours)	~14M	~73%

SHREK-Large (~83%) outperforms Original HRM (~75%) by 8 percentage points. SHREK-Small (~73%) matches the 27M-parameter Original HRM with roughly half the parameters. SHREK-Large nearly matches TRM Attention (~83% vs ~87%), closing the gap to within 4 percentage points.

C. Ablation Study

To isolate the contribution of each SHREK component, we conducted an ablation study on SHREK-Large using the Sudoku-Extreme dataset. The two novel components are: (1) error injection, which combines flip rate and learned error estimator signals injected into $\mathbf{z}_H$, and (2) stagnation delta, which feeds a measure of $\mathbf{z}_H$ change to the Q-head for halt decisions. All ablation variants include EMA to control for its known stabilizing effect. Results are shown in Table IV. Error injection is the dominant contribution (+10% over Original HRM vs +7% for stagnation delta alone). The combined improvement (+12%) exceeds either component in isolation.

TABLE IV
ABLATION OF SHREK COMPONENTS (SHREK-LARGE, SUDOKU).

Configuration	Err. Inj.	Stag. Δ	Accuracy	vs HRM
Original HRM	No	No	53%	(base)
HRM + EMA	No	No	~57%	+4%
Only stagnation delta	No	Yes	~60%	+7%
Only error injection	Yes	No	~63%	+10%
SHREK-Large (full)	Yes	Yes	~65%	+12%

D. Computational Efficiency

To address the gap of missing real-world cost comparisons, we measured inference GFLOPS on GPU using PyTorch’s FlopCounterMode over 1,000 test puzzles, using the final checkpoint of each training run. Tables V and VI report three values per model. Steps is the average number of reasoning iterations before the model halts, determined from the Q-head

logits. GF/step is the GFLOPS cost of a single reasoning step. GF/puzzle is the total compute per puzzle, computed as GF/step $\times$ Steps. The accuracy column reports the best-checkpoint test accuracy from the same training run, as in Tables II and III.

TABLE V
COMPUTATIONAL COST ON SUDOKU-EXTREME.

Model	Params	Steps	GF/step	GF/puzzle	Acc.
SHREK-Small(Ours)	$\sim 14\text{M}$	9.9	6.71	66.7	$\sim 63\%$
SHREK-Large(Ours)	$\sim 27\text{M}$	8.2	13.41	110.4	$\sim 65\%$
Original HRM	$\sim 27\text{M}$	10.7	13.41	143.5	53%
TRM Attention	$\sim 7\text{M}$	5.9	28.58	168.6	$\sim 70\%$

TABLE VI
COMPUTATIONAL COST ON MAZE-HARD.

Model	Params	Steps	GF/step	GF/puzzle	Acc.
TRM Att.	$\sim 7\text{M}$	1.2	238.85	274.7	$\sim 87\%$
SHREK-Small(Ours)	$\sim 14\text{M}$	5.6	73.70	409.8	$\sim 73\%$
Original HRM	$\sim 27\text{M}$	6.7	147.39	983.1	$\sim 75\%$
SHREK-Large(Ours)	$\sim 27\text{M}$	10.3	147.39	1519.6	$\sim 83\%$

On Sudoku-Extreme, SHREK-Small uses the fewest GFLOPs per puzzle (67) at 9.9 steps, followed by SHREK-Large (110 GFLOPs, 8.2 steps). Original HRM requires 144 GFLOPs at 10.7 steps. TRM Attention use fewer steps (5.9) but have higher per-step cost, resulting in 169 GFLOPs. SHREK-Large achieves 12 percentage points higher accuracy than Original HRM while using 23% fewer GFLOPs.

On Maze-Hard, TRM Attention uses 275 GFLOPs at only 1.2 steps. SHREK-Small uses 410 GFLOPs at 5.6 steps, making it more efficient than Original HRM (983 GFLOPs, 6.7 steps) while achieving comparable accuracy ($\sim 73\%$ vs $\sim 75\%$). SHREK-Large uses the most compute (1520 GFLOPs, 10.3 steps) but achieves 8 percentage points higher accuracy than Original HRM.

Figure 1 and Figure 2 visualize the accuracy versus efficiency trade-off across all models.

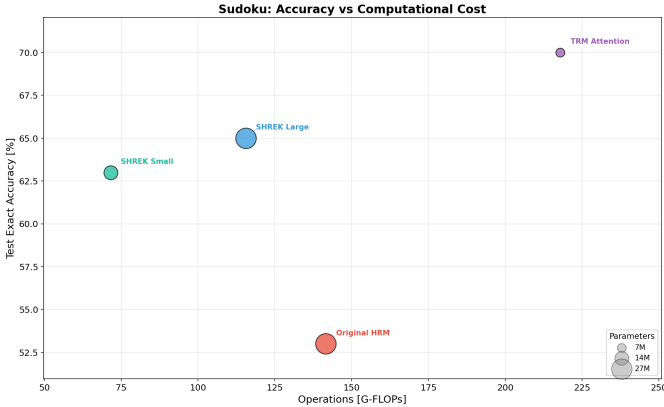


Fig. 1. Accuracy vs. GFLOPs per puzzle on Sudoku-Extreme. Bubble size represents parameter count.

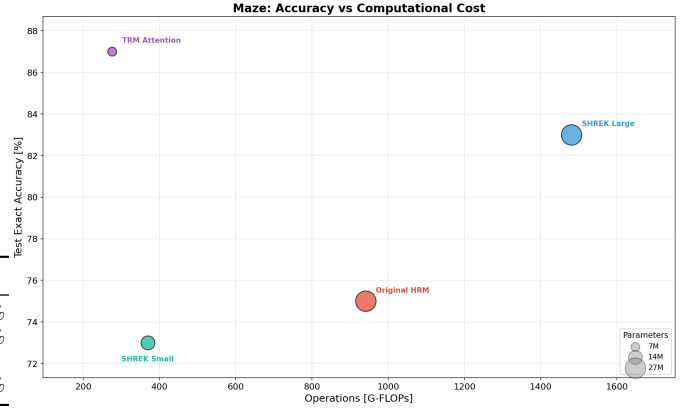


Fig. 2. Accuracy vs. GFLOPs per puzzle on Maze-Hard. Bubble size represents parameter count.

E. Training Time

Table VII reports wall-clock training time per model on a single NVIDIA GH200, recorded from SLURM accounting. Both SHREK variants train faster than TRM Attention on every benchmark, and SHREK-Small is the fastest model overall, completing each task in roughly half the time of SHREK-Large. On Maze-Hard, SHREK-Small finishes in 4h, half the wall-clock of both Original HRM (8h) and TRM Attention (7h), while still matching HRM’s accuracy with only 14M parameters. SHREK-Large matches Original HRM in training time on Maze-Hard despite adding the error_estimator and error_encoder modules. Across both datasets the SHREK variants therefore deliver an improved accuracy/compute trade-off relative to TRM Attention, and reach HRM’s accuracy or better at equal or lower training time.

TABLE VII
WALL-CLOCK TRAINING TIME PER MODEL AND DATASET (1 $\times$ NVIDIA GH200).

Model	Sudoku-Extreme	Maze-Hard
Original HRM	2h	8h
TRM Attention	7h	7h
SHREK-Small	2h	4h
SHREK-Large	4h	8h

F. Summary of Findings

Across both benchmarks, five key findings emerge:

- 1) **SHREK-Large outperforms Original HRM on both tasks:** +12% on Sudoku-Extreme and +8% on Maze-Hard.
- 2) **SHREK-Small (14M parameters) is competitive with 27M-parameter models:** it matches Original HRM on Maze-Hard and surpasses it on Sudoku-Extreme.
- 3) **Error injection is the primary SHREK contribution:** +10% over Original HRM in ablation on Sudoku-Extreme dataset.

- 4) **Computational efficiency:** On Sudoku-Extreme, both SHREK-Small and SHREK-Large are more computationally efficient than Original HRM and TRM Attention.
- 5) **Stagnation delta provides modest benefit:** +7% over Original HRM on Sudoku-Extreme.

VI. DISCUSSION

The results suggest that hierarchical reasoning models benefit more from internal feedback mechanisms than simply increasing parameter count. SHREK introduces self-correction through error injection and stagnation-aware halting, allowing the model to detect unstable reasoning states during inference. This helps address some of the main problems observed in HRM and TRM, such as oscillation and convergence to incorrect solutions [7]. The improvements from SHREK show that reasoning performance is strongly connected to how the model monitors and adjusts its internal reasoning process.

Another important observation is the relationship between reasoning quality and computational efficiency. On Sudoku-Extreme, SHREK achieves better accuracy while also reducing GFLOPs through earlier and more reliable halting. However, the Maze results show that improved reasoning does not always reduce computation, as SHREK sometimes requires more reasoning steps to reach better solutions. This suggests that different tasks have different reasoning dynamics, and that stronger reasoning may sometimes require additional compute.

Overall, the findings support the idea that architectural improvements and self-correction mechanisms can improve reasoning stability without relying on larger models. This also aligns with the goals of Green AI [4], where improving efficiency through better architectures may be more sustainable than scaling model size alone.

VII. CONCLUSION

This work examined whether hierarchical reasoning architectures can be improved with self-correction mechanisms to improve stability, accuracy, and computational efficiency without increasing parameter count. We introduced SHREK-HRM, which extends the original HRM architecture with a flip-rate detector, a learned error estimator, and stagnation-aware halting.

The results show that SHREK improves both accuracy and stability compared to the original HRM paper, even when using the same number of parameters. We found that SHREK-Small demonstrated that smaller models can remain competitive with larger ones when using reasoning mechanisms. This supports the idea that improvements in reasoning performance do not necessarily require an increase in the model scale.

A key insight from the model is that error injection plays a big role in these improvements. By combining the flip-rate signals with a learned error estimator, this gives SHREK an internal feedback that allows it to adjust its reasoning during inference.

We also see that self-correcting can improve computation efficiency, particularly on tasks like Sudoku-Extreme, where

earlier and more reliable halting reduces overall computation. We see from Maze-Hard that this depends on the task and it might also require more compute.

Our work shows that adding lightweight self-correcting mechanisms can improve hierarchical reasoning models. This aligns with Green AI [4], showing that better performance can be achieved through architectural improvements rather than increased scale, with a focus on more efficient and sustainable approach to building reasoning systems.

A. Limitations

There are multiple limitations with our solution. The first limitation is that the evaluation is limited to symbolic reasoning tasks such as Sudoku-Extreme and Maze-Hard. These problems are useful for studying structured reasoning, but they do not necessarily reflect performance on real-world problems.

The second constraint is that the computational results show that self-correction does not consistently reduce the cost across all tasks. While SHREK improves efficiency on Sudoku, it requires more reasoning steps on Maze-Hard, suggesting that the current halting mechanism is not fully task-adaptive.

Finally, no extensive hyperparameter search was performed. The models were trained using settings based on prior work, meaning that the reported results may not represent the best possible performance of SHREK-HRM.

B. Future Work

For future work, there are different ways this can be further researched. First, SHREK-HRM should be evaluated on a broader range of benchmarks, including ARC-AGI and natural language reasoning tasks, to test whether the self-correction mechanisms generalize beyond structured symbolic problems.

Second, improving the halting mechanism is an important next step. The current results show that SHREK can sometimes use more reasoning steps on certain tasks, suggesting that a more task-adaptive halting strategy could improve both efficiency and stability.

Another direction is to explore more advanced error estimation methods. Although the current learned error estimator is simple and effective, more expressive models could potentially provide better signals to guide the reasoning process.

In addition, future work should include more controlled comparisons under identical hardware conditions, as well as multi-GPU training to better match the setups used in prior work. This would allow for more reliable benchmarking and stronger conclusions.

Finally, a possible direction for SHREK is to measure real energy consumption as this would provide a more complete evaluation in accordance with the goals of Green AI [4].

REFERENCES

- [1] C. Cuskley, R. Woods, and M. Flaherty, "The limitations of large language models for understanding human language and cognition," *Open Mind*, vol. 8, pp. 1058–1083, 2024. [Online]. Available: https://doi.org/10.1162/opmi_a_00160
- [2] K. Valmeekam, S. Sreedharan, M. Marquez, A. Olmo, and S. Kambhampati, "Large language models still can't plan (a benchmark for llm planning)," *arXiv preprint arXiv:2308.06530*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.06530>

- [3] B. M. Lake and M. Baroni, “Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 2873–2882. [Online]. Available: <https://proceedings.mlr.press/v80/lake18a.html>
- [4] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green ai,” 2019. [Online]. Available: <https://arxiv.org/abs/1907.10597>
- [5] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori, “Hierarchical reasoning model,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.21734>
- [6] A. Jolicoeur-Martineau, “Less is more: Recursive reasoning with tiny networks,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.04871>
- [7] Z. Ren and Z. Liu, “Are your reasoning models reasoning or guessing? a mechanistic analysis of hierarchical reasoning models,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.10679>
- [8] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.15556>

APPENDIX

A. Declaration of AI usages

AI tools were utilized for language refinement and structural formatting of the manuscript. It is used to get feedback and help us refine the paper, not as a substitute for writing. We also used AI tools in the coding part of the project. We automatized git commit messages and used them to streamline the process by removing repetitive work. We used Grok, Gemini, Chat-GPT, and Claude.

B. Individual contributions

- Max contributed to the project 80%.
- Thomas contributed to the project 100%.
- Sebastian contributed to the project 80%.

In the beginning we did pair programming where all three of us programmed mostly together. We used Thomas computer, since he had the Simula connection. We did some individual work, but the main part was conducted in a group setting. Later on Thomas did more than work on the code, solving bugs and fixes to finish the work. He also wrote on the Results part in the paper. Max wrote more about the Research Gaps, Objective and Research Question. As well as the Method, Proposed Solution and Experiment in the Paper. Sebastian wrote the introduction, abstract, discussion, conclusion, limitations and future work in the paper. Otherwise we worked together finishing up the code delivered and the paper. Rewrite and refinement for the final delivery of the exam. The reason we give Thomas a bit higher score is because he contributed a bit more on the coding part of the project, and this work being harder and more time consuming.